

MAIE 5101 Programming for Artificial Intelligence

Final Exam Notes: Lectures #1–#6

Spring 2026 | HKUST CSE

Instructors: Dr. Cecia Chan & Dr. Desmond Tsoi

Contents

1	Final Exam Information and Scope	4
1.1	Exam Logistics	4
1.2	Exam Coverage	4
2	Lecture #1: Python Basics, Flow Control, and Collections	5
2.1	Python Basics	5
2.1.1	Python as a Language	5
2.1.2	Program Structure	5
2.1.3	Basic Data Types	5
2.1.4	Container Types	5
2.1.5	Mutability	6
2.1.6	Variables and Assignment	6
2.1.7	Strings and Formatting	6
2.1.8	Input, Output, and Type Conversion	6
2.1.9	Operators	6
2.1.10	Comparison, Logical, Identity, and Membership Operators	7
2.1.11	Operator Precedence	7
2.1.12	Floating Point Errors	7
2.2	Flow Control	7
2.2.1	Branching	7
2.2.2	<code>match-case</code>	8
2.2.3	<code>while</code> Loop	8
2.2.4	<code>for</code> Loop and <code>range()</code>	8
2.2.5	<code>break</code> , <code>continue</code> , and Loop-else	8
2.3	Collections	9
2.3.1	Collection Comparison	9
2.3.2	Common Sequence Operations	9
2.3.3	Lists	9
2.3.4	List Comprehensions	9
2.3.5	Tuples	9
2.3.6	Dictionaries	10
2.3.7	Sets	10
2.3.8	Strings as Sequences	10
2.3.9	Copying	10
3	Lecture #2: Functions, Recursion, and Object-Oriented Programming	12
3.1	Functions	12
3.1.1	Defining and Calling Functions	12
3.1.2	Arguments and Parameters	12
3.1.3	Mutable Default Argument	12
3.1.4	<code>*args</code> and <code>**kwargs</code>	12

3.1.5	Returning Multiple Values	13
3.1.6	Parameter Passing and Mutability	13
3.1.7	Scope and LEGB Rule	13
3.1.8	<code>nonlocal</code>	13
3.1.9	Lambda, <code>map()</code> , <code>filter()</code> , and <code>reduce()</code>	13
3.2	Recursion	14
3.2.1	Basic Structure	14
3.2.2	Factorial	14
3.2.3	Permutation Backtracking Pattern	14
3.3	Decorators and Closures	15
3.3.1	Closures	15
3.3.2	Decorators	15
3.4	Object-Oriented Programming	15
3.4.1	Classes and Objects	15
3.4.2	Access Conventions	16
3.4.3	Instance Attributes vs. Class Attributes	16
3.4.4	Methods	16
3.4.5	<code>@property</code>	16
3.4.6	Operator Overloading	17
3.4.7	Inheritance and Polymorphism	17
4	Lectures #3–#4: NumPy	18
4.1	Array Creation and Properties	18
4.2	Indexing and Slicing	18
4.3	Fancy Indexing and Boolean Masking	18
4.4	Views and Copies	19
4.5	Element-wise Operations and Matrix Multiplication	19
4.6	Aggregation and Axis	19
4.7	Reshaping and Transposing	19
4.8	Broadcasting	20
4.9	Vectorized Computation Pattern	20
4.10	NumPy vs. Python Lists	20
5	Lecture #5: Pandas Basics	21
5.1	Series	21
5.2	DataFrame	21
5.3	Column Selection	21
5.4	<code>loc</code> and <code>iloc</code>	21
5.5	Boolean Filtering	22
5.6	Basic Operations	22
5.7	Basic Aggregation	22
5.8	File Input and Output	22
6	Lecture #6: Data Analysis and Preprocessing using Pandas	23
6.1	Handling Missing Data	23
6.2	Removing Duplicates	23
6.3	Replacing Values	23
6.4	Applying Functions	23
6.5	Discretization	23
6.6	One-Hot Encoding	24
6.7	Filtering and Sorting	24
6.8	Combining DataFrames	24

6.8.1	Concatenation	24
6.8.2	Merge and Join	24
6.9	GroupBy: Split-Apply-Combine	24
6.9.1	as_index=False	24
6.9.2	agg(), transform(), and apply()	25
6.10	Pandas Exam Patterns	25
6.10.1	Boolean Filtering with loc	25
6.10.2	GroupBy with iloc	25
Appendix: Quick Reference for Final Exam		26
Sample Exam Analysis		28

1 Final Exam Information and Scope

1.1 Exam Logistics

- **Date:** 18 May 2026, Monday.
- **Time:** 7:00pm–9:00pm.
- **Duration:** 2 hours.
- **Venue:** LG3008 / LG3009, Lift 10–12.
- **Seat Assignment:** Seating plan available from 18 May 2026, noon.
- **Format:** Closed-book and closed-note.
- **Allowed Material:** One double-sided cheat sheet.
- **Allowed Device:** HKEAA-approved calculator only.
- **Question Types:** True/False, Multiple Choice, Short Questions, Long Questions.

Common Pitfall / Exam Trap

No phones, smart watches, smart glasses, tablets, computers, or other electronic devices are allowed, except approved calculators. Bring the HKUST student ID card. Late arrival gives no extra time, and there is no make-up exam.

1.2 Exam Coverage

Lecture	Topics Covered in Final Exam
Lecture #1	Python Basics, Flow Control, Collections
Lecture #2	Functions and Recursion, Object-Oriented Programming
Lectures #3–#4	Scientific Computing using Numerical Python, NumPy
Lecture #5	Pandas Basics
Lecture #6	Data Analysis and Preprocessing using Pandas

Key Point

The final exam covers **Lecture #1 to Lecture #6 only**. Therefore, Matplotlib, SciPy, Scikit-learn, NLP, Image Processing, Neural Networks, PyTorch, TensorFlow, Transfer Learning, Flask, and Docker are removed from these final-exam notes.

2 Lecture #1: Python Basics, Flow Control, and Collections

2.1 Python Basics

2.1.1 Python as a Language

Python is a **high-level**, **interpreted**, **dynamically typed**, and **object-oriented** programming language.

- **Interpreted:** Code is executed by the Python interpreter.
- **Dynamically typed:** Variable types are determined at runtime.
- **Object-oriented:** Values are objects with identity, type, and value.
- **Cross-platform:** Python programs can run on different operating systems.

2.1.2 Program Structure

- Python does not require semicolons at the end of statements.
- **Indentation** defines code blocks.
- Statements ending with `:` expect an indented block.
- Multi-line statements can be written inside `()`, `[]`, or `{}`.

```

1 def main():
2     print("Hello")
3
4 if __name__ == "__main__":
5     main()

```

2.1.3 Basic Data Types

Name	Type	Examples
Integer	<code>int</code>	3, -200, 0
Float	<code>float</code>	4.3, 5., 2.23e-3
Complex	<code>complex</code>	1+2j
Boolean	<code>bool</code>	True, False
String	<code>str</code>	"Hello", 'world'
None	<code>NoneType</code>	None

2.1.4 Container Types

Name	Type	Ordered?	Mutable?
List	<code>list</code>	Yes	Yes
Tuple	<code>tuple</code>	Yes	No
Dictionary	<code>dict</code>	Yes, Python 3.7+	Yes
Set	<code>set</code>	No	Yes
Frozen Set	<code>frozenset</code>	No	No

Common Pitfall / Exam Trap

A **set** does not maintain insertion order. Dictionary keys must be hashable, so lists and dictionaries cannot be used as dictionary keys.

2.1.5 Mutability

- **Mutable:** list, dict, set
- **Immutable:** int, float, complex, bool, str, tuple, frozenset, None

2.1.6 Variables and Assignment

```
1 x = 10
2 a = b = c = 1
3 x, y = 3, 4
4 x, y = y, x
```

- Variable names are case-sensitive.
- Variables can be reassigned to values of different types.
- Type hints are not enforced at runtime.

```
1 name: str = "Alice"
2 age: int = 20
```

2.1.7 Strings and Formatting

```
1 name = "Alice"
2 score = 95.12345
3 print(f"{name}: {score:.2f}")
```

Common string prefixes:

Prefix	Meaning	Example
r	Raw string	r"\n"
f	Formatted string	f"{x}"
b	Byte string	b"abc"

2.1.8 Input, Output, and Type Conversion

```
1 age = input("Enter your age: ") # always returns str
2 age = int(age)
3 print(age)
```

Useful conversion functions: `int()`, `float()`, `str()`, `bool()`.

Common Pitfall / Exam Trap

`input()` always returns a string. `int(52.7)` returns 52; it truncates rather than rounds.

2.1.9 Operators

Operator	Meaning	Example
+, -, *	Arithmetic	3 + 4
/	Float division	5 / 2 gives 2.5
//	Floor division	5 // 2 gives 2
%	Modulo	5 % 2 gives 1
**	Exponentiation	2 ** 3 gives 8

Key Point

The `/` operator always returns a float. The `**` operator is right-associative, so `2**3**2` means `2**(3**2)`.

Modulo formula:

$$r = a - (a // b) \times b$$

Example:

$$-10 \bmod 3 = -10 - (-4) \times 3 = 2$$

2.1.10 Comparison, Logical, Identity, and Membership Operators

- Comparison: `<`, `<=`, `>`, `>=`, `==`, `!=`
- Logical: `not`, `and`, `or`
- Identity: `is`, `is not`
- Membership: `in`, `not in`

Common Pitfall / Exam Trap

Logical operators use short-circuit evaluation. For example, `True or 2/0` does not raise an error because the second expression is not evaluated.

2.1.11 Operator Precedence

Operator	Associativity
<code>**</code>	Right-to-left
Unary <code>+</code> , <code>-</code>	Right-to-left
<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Left-to-right
<code>+</code> , <code>-</code>	Left-to-right
Comparisons, <code>is</code> , <code>in</code>	Left-to-right
<code>not</code>	Left-to-right
<code>and</code>	Left-to-right
<code>or</code>	Left-to-right

2.1.12 Floating Point Errors

```
1 0.1 + 0.2 == 0.3    # False
```

Use `math.isclose()` when comparing floating-point numbers.

2.2 Flow Control**2.2.1 Branching**

```
1 if condition1:
2     ...
3 elif condition2:
4     ...
5 else:
6     ...
```

Ternary expression:

```
1 result = "even" if n % 2 == 0 else "odd"
```

2.2.2 match-case

Keep this topic only if it appeared in the lecture slides.

```
1 match value:
2     case 1:
3         print("one")
4     case 2 | 3:
5         print("two or three")
6     case _:
7         print("other")
```

2.2.3 while Loop

```
1 counter = 0
2 while counter < 10:
3     print(counter)
4     counter += 1
```

Common Pitfall / Exam Trap

A loop must eventually make its condition false; otherwise it becomes an infinite loop.

2.2.4 for Loop and range()

```
1 for i in range(5):
2     print(i)          # 0, 1, 2, 3, 4
```

Exam Tip

range(n) produces values from 0 to n-1. The stop value is excluded.

2.2.5 break, continue, and Loop-else

- break: exits the innermost loop.
- continue: skips to the next iteration.
- Loop-else: runs only if the loop finishes normally without break.

```
1 for i in range(2, n):
2     if n % i == 0:
3         print("not prime")
4         break
5 else:
6     print("prime")
```

2.3 Collections

2.3.1 Collection Comparison

Feature	str	list	tuple	dict	set	frozenset
Ordered	Yes	Yes	Yes	Yes	No	No
Mutable	No	Yes	No	Yes	Yes	No
Indexed	Yes	Yes	Yes	By key	No	No
Sliceable	Yes	Yes	Yes	No	No	No
Duplicates	Yes	Yes	Yes	Keys: No	No	No
Hashable	Yes	No	If all elements	No	No	Yes

2.3.2 Common Sequence Operations

Operation	Description
<code>x in s</code>	Membership test
<code>s1 + s2</code>	Concatenation
<code>s * n</code>	Repetition
<code>s[i]</code>	Indexing
<code>s[i:j:k]</code>	Slicing
<code>len(s)</code>	Length

2.3.3 Lists

```

1 lst = [1, 2, 3]
2 lst.append(4)
3 lst.extend([5, 6])
4 lst.insert(0, 0)
5 lst.remove(3)
6 x = lst.pop()
7 lst.sort()
8 lst.reverse()

```

Common Pitfall / Exam Trap

`append(x)` adds `x` as one element. `extend(iterable)` iterates through its argument.

```

1 lst = [1, 2]
2 lst.append("34")      # [1, 2, '34']
3
4 lst = [1, 2]
5 lst.extend("34")      # [1, 2, '3', '4']

```

2.3.4 List Comprehensions

```

1 squares = [x**2 for x in range(5)]
2 evens = [x for x in range(20) if x % 2 == 0]

```

2.3.5 Tuples

```

1 t = (1, 2, 3)
2 single = (1,)      # single-element tuple
3 a, b, c = t         # unpacking

```

Common Pitfall / Exam Trap

A tuple is immutable, but a mutable object inside a tuple can still be modified.

2.3.6 Dictionaries

```

1 d = {"a": 1, "b": 2}
2 d["c"] = 3
3 print(d["a"])
4 print(d.get("z", 0))
5 d.update({"d": 4})
6 d.pop("b")

```

Useful methods: `keys()`, `values()`, `items()`, `get()`, `pop()`, `update()`.

Common Pitfall / Exam Trap

Do not add or delete dictionary keys while iterating over the dictionary. It can raise `RuntimeError`.

2.3.7 Sets

```

1 s1 = {1, 2, 3}
2 s2 = {3, 4, 5}
3
4 s1 | s2      # union
5 s1 & s2      # intersection
6 s1 - s2      # difference
7 s1 ^ s2      # symmetric difference

```

2.3.8 Strings as Sequences

Strings are immutable sequences.

```

1 text = " hello world "
2 text.strip()
3 text.upper()
4 text.split()
5 "-".join(["a", "b", "c"])
6 text.replace("world", "Python")
7 text.find("hello")

```

2.3.9 Copying

```

1 import copy
2
3 a = [[1, 2], [3, 4]]
4 b = a          # reference
5 c = a.copy()   # shallow copy
6 d = copy.deepcopy(a) # deep copy

```

Key Point

Assignment copies references, not objects. Shallow copy copies only the outer container.
Deep copy recursively copies nested objects.

3 Lecture #2: Functions, Recursion, and Object-Oriented Programming

3.1 Functions

3.1.1 Defining and Calling Functions

```

1 def function_name(param1, param2):
2     """Optional docstring."""
3     return param1 + param2

```

A function without an explicit `return` returns `None`.

3.1.2 Arguments and Parameters

```

1 def print_area(width=1, height=2):
2     print(width * height)
3
4 print_area(4, 2)
5 print_area(width=3)

```

Key Point

Default parameters must appear after required parameters. Positional arguments cannot appear after keyword arguments.

3.1.3 Mutable Default Argument

Common Pitfall / Exam Trap

Mutable default arguments are created once at function definition time. Mutations persist across calls.

```

1 def add(x, y=[]):
2     y.append(x)
3     return y
4
5 print(add(1))      # [1]
6 print(add(2))      # [1, 2]

```

Correct pattern:

```

1 def add(x, y=None):
2     if y is None:
3         y = []
4     y.append(x)
5     return y

```

3.1.4 `*args` and `**kwargs`

```

1 def demo(*args, **kwargs):
2     print(args)      # tuple
3     print(kwargs)    # dict

```

3.1.5 Returning Multiple Values

```
1 def sort_two(a, b):
2     return (a, b) if a < b else (b, a)
3
4 small, large = sort_two(5, 3)
```

3.1.6 Parameter Passing and Mutability

Python uses pass-by-object-reference.

```
1 def modify(numbers):
2     numbers[0] = 999
3
4 x = [1, 2, 3]
5 modify(x)
6 print(x)      # [999, 2, 3]
```

Key Point

Reassigning an immutable object inside a function does not affect the caller. Mutating a mutable object in place does affect the caller.

3.1.7 Scope and LEGB Rule

LEGB means Local, Enclosing, Global, Built-in.

```
1 x = 10
2
3 def increase():
4     global x
5     x += 1
```

Common Pitfall / Exam Trap

If a function assigns to a variable name anywhere inside the function, Python treats that name as local throughout the function. Reading it before assignment causes `UnboundLocalError`.

3.1.8 nonlocal

```
1 def outer():
2     count = 0
3     def inner():
4         nonlocal count
5         count += 1
6         return count
7     return inner
```

3.1.9 Lambda, map(), filter(), and reduce()

```

1 square = lambda x: x ** 2
2
3 list(map(lambda s: len(s), ["apple", "banana"]))
4 list(filter(lambda x: x % 2 == 0, range(10)))
5
6 from functools import reduce
7 reduce(lambda a, b: a + b, [1, 2, 3, 4])

```

Common Pitfall / Exam Trap

Lambda functions capture variables by reference. The variable is looked up when the function is called, not when it is defined.

```

1 funcs = [lambda x: x * i for i in range(3)]
2 print([f(2) for f in funcs])      # [4, 4, 4]
3
4 funcs = [lambda x, i=i: x * i for i in range(3)]
5 print([f(2) for f in funcs])      # [0, 2, 4]

```

3.2 Recursion

3.2.1 Basic Structure

Key Point

Every recursive solution needs:

1. a base case,
2. a recursive call on a smaller problem,
3. a way to combine or return the result.

```

1 def solve(problem):
2     if base_case(problem):
3         return base_solution
4     return solve(smaller_problem)

```

3.2.2 Factorial

```

1 def factorial(n):
2     if n == 0:
3         return 1
4     return n * factorial(n - 1)

```

3.2.3 Permutation Backtracking Pattern

```

1 def permutations(input_str):
2     def backtrack(chars, index):
3         if index == len(chars):
4             results.append("".join(chars))
5             return
6
7         for i in range(index, len(chars)):

```

```

8         chars[index], chars[i] = chars[i], chars[index]
9         backtrack(chars, index + 1)
10        chars[index], chars[i] = chars[i], chars[index]
11
12    results = []
13    backtrack(list(input_str), 0)
14    return results

```

3.3 Decorators and Closures

This section is lower priority than basic functions, recursion, and OOP, but it appears in Lecture #2 additional materials.

3.3.1 Closures

A closure is created when an inner function remembers variables from its enclosing function.

```

1 def create_averager():
2     values = []
3     def averager(num):
4         values.append(num)
5         return sum(values) / len(values)
6     return averager

```

3.3.2 Decorators

A decorator is a function that takes a function and returns another function.

```

1 from functools import wraps
2
3 def show_running(func):
4     @wraps(func)
5     def wrapper(*args, **kwargs):
6         print(f"Running: {func.__name__}")
7         return func(*args, **kwargs)
8     return wrapper
9
10 @show_running
11 def is_even(n):
12     return n % 2 == 0

```

3.4 Object-Oriented Programming

3.4.1 Classes and Objects

A class is a blueprint. An object is an instance of a class. Each object has identity, type, and value.

```

1 import math
2
3 class Circle:
4     def __init__(self, radius):
5         self.radius = radius
6
7     def area(self):
8         return math.pi * self.radius ** 2

```

3.4.2 Access Conventions

Convention	Prefix	Meaning	Mechanism
Public	none	Accessible anywhere	Default
Protected	_	Internal use by convention	Convention only
Private	--	Name mangling	Not true privacy

Common Pitfall / Exam Trap

`--attr` is not truly private. It is name-mangled to `_ClassName__attr`.

3.4.3 Instance Attributes vs. Class Attributes

```

1 class Person:
2     count = 0
3
4     def __init__(self, name):
5         self.name = name
6         Person.count += 1

```

Common Pitfall / Exam Trap

Assigning to an instance attribute can shadow a class attribute. Other instances and the class attribute itself are not necessarily changed.

3.4.4 Methods

Method Type	First Parameter	Instance Access?	Class Access?
Instance method	<code>self</code>	Yes	Yes
Class method	<code>cls</code>	No	Yes
Static method	<code>none</code>	No	No

3.4.5 @property

```

1 class Model:
2     def __init__(self, name):
3         self.__name = name
4
5     @property
6     def model_info(self):
7         return self.__name
8
9 m = Model("GPT")
10 print(m.model_info)      # no parentheses

```

3.4.6 Operator Overloading

Operator	Method	Example
+	<code>--add--</code>	<code>a + b</code>
<code>==</code>	<code>--eq--</code>	<code>a == b</code>
<code><</code>	<code>--lt--</code>	<code>a < b</code>
<code>str()</code>	<code>--str--</code>	<code>print(a)</code>
<code>len()</code>	<code>--len--</code>	<code>len(a)</code>

3.4.7 Inheritance and Polymorphism

```
1 class Student(Person):
2     def __init__(self, name, sid):
3         super().__init__(name)
4         self.sid = sid
5
6     def print_info(self):
7         print(self.name, self.sid)
```

Key Point

Use `super()` to call methods from the parent class. Method overriding allows subclasses to provide specialized behavior.

4 Lectures #3–#4: NumPy

4.1 Array Creation and Properties

```

1 import numpy as np
2
3 a = np.array([10, 20, 30])
4 b = np.array([[1, 2, 3], [4, 5, 6]])
5
6 np.zeros((3, 3))
7 np.ones((2, 3))
8 np.full((3, 2), 7)
9 np.eye(4)
10 np.arange(0, 10, 2)
11 np.linspace(0, 1, 5)
12 np.random.randn(3, 2)

```

Key attributes:

- **ndim**: number of dimensions
- **shape**: array shape
- **dtype**: data type
- **size**: number of elements

4.2 Indexing and Slicing

```

1 a = np.array([[10, 20, 30, 40],
2               [50, 60, 70, 80],
3               [90, 100, 110, 120]])
4
5 a[0, 2]          # 30
6 a[:2, 2:4]       # shape (2, 2)
7 a[2, :]          # shape (4,)
8 a[2:3, :]        # shape (1, 4)

```

Common Pitfall / Exam Trap

Integer indexing reduces rank. Slicing preserves rank. Therefore, `a[2, :]` and `a[2:3, :]` have different shapes.

4.3 Fancy Indexing and Boolean Masking

```

1 a = np.array([[1, 2], [3, 4], [5, 6]])
2
3 a[[0, 1, 2], [1, 0, 1]]      # [2, 3, 6]
4 a[a > 3]                     # [4, 5, 6]
5 a[(a > 1) & (a < 5)]         # [2, 3, 4]

```

Common Pitfall / Exam Trap

Use parentheses around boolean conditions combined with `&`, `|`, or `~`.

4.4 Views and Copies

Key Point

`b = a` creates an alias. Basic slicing usually returns a view. Fancy indexing, boolean indexing, `copy()`, and `flatten()` return copies.

```

1 a = np.array([1, 2, 3])
2 b = a
3 b[0] = 99
4 print(a)           # [99, 2, 3]
5
6 c = a.copy()
7 c[0] = 100
8 print(a)           # unchanged by c

```

4.5 Element-wise Operations and Matrix Multiplication

```

1 x + y           # element-wise addition
2 x * y           # element-wise multiplication
3 x @ y           # matrix multiplication
4 np.dot(v, w)
5 np.matmul(x, y)

```

Common Pitfall / Exam Trap

`x * y` is element-wise multiplication, not matrix multiplication. Use `@` or `np.matmul()` for matrix multiplication.

4.6 Aggregation and Axis

```

1 a = np.array([[1, 2], [3, 4]])
2
3 np.sum(a)           # 10
4 np.sum(a, axis=0)   # [4, 6]
5 np.sum(a, axis=1)   # [3, 7]
6
7 np.mean(a)
8 np.std(a)
9 np.max(a)
10 np.argmax(a)

```

Key Point

For a 2D array, `axis=0` collapses rows and gives per-column results. `axis=1` collapses columns and gives per-row results.

4.7 Reshaping and Transposing

```

1 a = np.arange(12).reshape(3, 4)
2 a.T
3

```

```

4 v = np.array([1, 2, 3])
5 v[:, np.newaxis]      # shape (3, 1)
6 v[np.newaxis, :]      # shape (1, 3)

```

4.8 Broadcasting

Broadcasting allows element-wise operations on arrays with different but compatible shapes.

1. Compare shapes from right to left.
2. Dimensions are compatible if they are equal or one of them is 1.
3. If ranks differ, prepend 1s to the smaller-rank shape.

```

1 x = np.array([[1, 2, 3],
2               [4, 5, 6],
3               [7, 8, 9]])
4
5 v = np.array([1, 0, 1])
6 y = x + v

```

Common Pitfall / Exam Trap

`np.array([1,2,3])` has shape `(3,)`. `np.array([[4],[5],[6]])` has shape `(3,1)`. Adding them broadcasts to shape `(3,3)`.

4.9 Vectorized Computation Pattern

```

1 R = np.std(np.mean(
2     np.where(X > c[np.newaxis, :],
3             X * np.sin(X),
4             X * np.cos(X)),
5     axis=0))

```

Key Point

If `X` has shape `(N, D)` and `c` has shape `(D,)`, then `X > c` broadcasts correctly. Using `c[np.newaxis, :]` makes the intended shape `(1, D)` explicit.

4.10 NumPy vs. Python Lists

	Python list	NumPy array
Element types	Mixed	Homogeneous
<code>a + b</code>	Concatenation	Element-wise addition
<code>a * 3</code>	Repetition	Element-wise multiplication
Slicing returns	Copy-like behavior	Usually view
Speed	Slower	Faster, vectorized

5 Lecture #5: Pandas Basics

5.1 Series

A Series is a one-dimensional labeled array.

```
1 import pandas as pd
2
3 s = pd.Series([10, 20, 30], index=["a", "b", "c"])
4
5 s["b"]          # label-based
6 s.iloc[1]       # position-based
```

5.2 DataFrame

A DataFrame is a two-dimensional labeled table.

```
1 df = pd.DataFrame({
2     "Name": ["Alice", "Bob", "Charlie"],
3     "Age": [28, 32, 27],
4     "City": ["Beijing", "Shanghai", "Guangzhou"]
5 })
```

Useful attributes:

- `df.shape`
- `df.dtypes`
- `df.columns`
- `df.index`

5.3 Column Selection

```
1 df["Name"]          # Series
2 df[["Name"]]        # DataFrame
3 df[["Name", "Age"]] # DataFrame
```

Common Pitfall / Exam Trap

`df["col"]` returns a **Series**. `df[["col"]]` returns a **DataFrame**.

5.4 loc and iloc

```
1 df.loc["a", "Name"] # label-based
2 df.iloc[0, 1]       # position-based
3
4 df.loc["a":"c"]     # end label included
5 df.iloc[0:3]        # end position excluded
```

Common Pitfall / Exam Trap

`loc` is label-based and includes the end label in slicing. `iloc` is position-based and excludes the end position.

5.5 Boolean Filtering

```
1 mask = (df["Age"] >= 30) & (df["City"] == "Shanghai")
2 df[mask]
```

Common Pitfall / Exam Trap

Use parentheses around each condition when combining boolean masks with `&` or `|`.

5.6 Basic Operations

```
1 df.head()
2 df.tail()
3 df.info()
4 df.describe()
5 df["City"].value_counts()
6
7 df.sort_values("Age")
8 df.sort_index()
```

5.7 Basic Aggregation

```
1 df["Age"].mean()
2 df["Age"].sum()
3 df["Age"].count()
4 df["Age"].min()
5 df["Age"].max()
```

Common Pitfall / Exam Trap

Pandas aggregation functions such as `mean()` skip missing values by default. They do not treat missing values as zero.

5.8 File Input and Output

```
1 df = pd.read_csv("data.csv", na_values=["N/A", "Missing"])
2 df.to_csv("output.csv", index=False)
```

Exam Tip

For a closed-book written exam, focus on reading, predicting output, indexing behavior, shape, and aggregation results rather than Jupyter Notebook interface details.

6 Lecture #6: Data Analysis and Preprocessing using Pandas

6.1 Handling Missing Data

```
1 df.isna()
2 df.notna()
3 df.isna().sum()
4
5 df.dropna()
6 df.fillna(0)
7 df["Score"].fillna(df["Score"].mean())
8 df["Score"].interpolate()
```

Key Point

Use `isna()` to detect missing values, `dropna()` to remove rows or columns with missing values, and `fillna()` to replace missing values.

6.2 Removing Duplicates

```
1 df.duplicated()
2 df.drop_duplicates()
3 df.drop_duplicates(subset=["Age", "Score"], keep="first")
```

6.3 Replacing Values

```
1 import numpy as np
2
3 df.replace({"Age": 0}, np.nan)
4 df["Gender"].replace({"M": "Male", "F": "Female"})
```

6.4 Applying Functions

```
1 df["Age"].map(lambda x: x + 1)
2 df["Score"].apply(lambda x: "Pass" if x >= 50 else "Fail")
3 df.apply(lambda row: row["Age"] + row["Score"], axis=1)
```

Key Point

`map()` is commonly used on a `Series`. `apply()` can be used on a `Series` or `DataFrame`. With `axis=1`, `apply()` processes rows.

6.5 Discretization

```
1 pd.cut(scores,
2       bins=[0, 40, 70, 100],
3       labels=["Low", "Medium", "High"])
4
5 pd.qcut(scores, q=4)
```

- `cut()` creates bins based on specified boundaries.
- `qcut()` creates bins with approximately equal numbers of observations.

6.6 One-Hot Encoding

```
1 pd.get_dummies(df, columns=["Gender"])
```

6.7 Filtering and Sorting

```
1 df[df["Age"] > 30]
2 df.loc[df["Score"] == df["Score"].max(), "Name"]
3
4 df.sort_values("Score")
5 df.sort_values(["Class", "Score"], ascending=[True, False])
```

6.8 Combining DataFrames

6.8.1 Concatenation

```
1 pd.concat([df1, df2])
2 pd.concat([df1, df2], ignore_index=True)
3 pd.concat([df1, df2], axis=1)
```

6.8.2 Merge and Join

```
1 pd.merge(df1, df2, on="key")
2 pd.merge(df1, df2, on="key", how="left")
3 pd.merge(df1, df2, on="key", how="right")
4 pd.merge(df1, df2, on="key", how="outer")
```

Method	Default Join	Join Key
pd.concat	Outer	Index or columns
pd.merge	Inner	Column values
df.join	Left	Row index

6.9 GroupBy: Split-Apply-Combine

```
1 grp = df.groupby("Class")
2
3 grp.mean()
4 grp["Score"].mean()
5 grp.agg(["mean", "std", "count"])
```

6.9.1 as_index=False

```
1 result = df.groupby("Class", as_index=False).sum()
```

Key Point

as_index=False keeps group keys as normal columns instead of using them as the index.

6.9.2 agg(), transform(), and apply()

```
1 df.groupby("Class")["Score"].agg(["mean", "max"])
2 df.groupby("Class")["Score"].transform("mean")
3 df.groupby("Class").apply(lambda g: g.nlargest(2, "Score"))
```

Key Point

Use `agg()` for summary tables. Use `transform()` when the result should align with the original DataFrame. Use `apply()` for flexible custom operations.

6.10 Pandas Exam Patterns

6.10.1 Boolean Filtering with loc

```
1 df.loc[df["Points"] == df["Points"].max(), "Player"]
```

6.10.2 GroupBy with iloc

```
1 result = df.groupby("Class", as_index=False).sum()
2 print(result.iloc[1, 0])
```

Common Pitfall / Exam Trap

When predicting Pandas output, carefully check whether the result is a **Series** or **DataFrame**, whether the index is preserved, and whether group keys become index values or columns.

Appendix: Quick Reference for Final Exam

Python Traps

1. `range(n)` goes from 0 to `n-1`.
2. `input()` always returns a string.
3. `/` always returns a float.
4. `//` is floor division.
5. `set` is unordered.
6. Dictionary keys must be hashable.
7. Lists and dictionaries are not hashable.
8. Assignment copies references, not objects.
9. Mutable default arguments persist across calls.
10. Lambda closures capture variables by reference.

Function and Recursion Traps

1. A function without `return` returns `None`.
2. Use `global` to reassign a global variable inside a function.
3. Use `nonlocal` to reassign an enclosing-scope variable.
4. Every recursion needs a base case.
5. Recursive calls must move toward the base case.

OOP Essentials

1. `self` refers to the current instance.
2. `__init__` initializes a new object.
3. Class attributes are shared unless shadowed by instance attributes.
4. `_name` is protected by convention only.
5. `__name` uses name mangling, not true privacy.
6. Use `super()` to call parent-class methods.
7. `@property` allows method access like an attribute.

NumPy Traps

1. `b = a` creates an alias.
2. Basic slicing usually returns a view.
3. Fancy indexing and boolean indexing return copies.
4. Integer indexing reduces rank.
5. Slicing preserves rank.
6. `*` means element-wise multiplication.
7. Use `@` or `np.matmul()` for matrix multiplication.
8. Broadcasting compares shapes from right to left.
9. Dimensions are compatible if equal or one of them is 1.

Pandas Traps

1. `df["col"]` returns a `Series`.
2. `df[["col"]]` returns a `DataFrame`.
3. `loc` is label-based and end-inclusive.
4. `iloc` is position-based and end-exclusive.
5. Boolean conditions need parentheses with `&` and `|`.
6. `mean()` skips missing values by default.

7. `groupby(..., as_index=False)` keeps group keys as columns.
8. `agg()` summarizes groups.
9. `transform()` returns values aligned with the original DataFrame.
10. `apply()` is flexible but usually slower.

Recursion Template

```
def solve(problem):  
    if base_case(problem):  
        return base_solution  
  
    smaller = make_smaller(problem)  
    result = solve(smaller)  
    return combine(problem, result)
```

Double-Sided Cheat Sheet Suggested Layout

- **Front side:** Python basics, flow control, collections, functions, recursion, OOP.
- **Back side:** NumPy shape rules, broadcasting, views/copies, Pandas indexing, missing data, merge, groupby.

Sample Exam Analysis

Overall Structure

Problem	Type	Points	Topics Tested
1	True/False (10 sub-questions)	10	Python, OOP, NumPy, Pandas
2	Multiple Choice (7 sub-questions)	10	Collections, Functions, OOP, NumPy, Pandas
3	Debugging (find 3 errors)	10	Functions, Data Types, Edge Cases
4	Recursion (write code)	10	Backtracking Permutation
5	OOP Fill-in-the-Blank (5 blanks)	10	Inheritance, <code>super()</code> , <code>@property</code>
6	NumPy Vectorization	10	Broadcasting, <code>np.where</code> , <code>Axis</code>
7	Pandas Output Prediction (2 parts)	5	<code>loc</code> , <code>groupby</code> , <code>iloc</code>
Total		65	

Exam Tip

The sample exam totals 65 points across 7 problems. The actual exam may differ in total points, number of questions, and difficulty. The exam instructions also mention Matplotlib code, suggesting the real exam may include plotting questions not shown in the sample.

Problem 1: True/False — Answers and Analysis

Q	Ans	Key Reasoning
(a)	F	<code>range(5)</code> produces 0–4; the final printed value is 4, not 5.
(b)	F	<code>set</code> does NOT maintain insertion order; only <code>dict</code> does (since 3.7+).
(c)	F	Dictionary keys must be hashable; lists and dicts are not hashable.
(d)	F	Default arguments are evaluated once at definition time, not each call.
(e)	F	<code>__attr</code> uses name mangling, not true privacy; still accessible via <code>_Class__attr</code> .
(f)	T	<code>Shape (3,) + shape (3,1)</code> broadcasts to <code>(3,3)</code> .
(g)	F	<code>b = a</code> creates an alias; modifying <code>b[0]</code> also changes <code>a</code> .
(h)	F	A Python list of booleans is interpreted as integer fancy indexing (<code>True</code> →1, <code>False</code> →0), not as a boolean mask. Only a NumPy boolean array works as a proper mask.
(i)	F	<code>.mean()</code> skips NaN (excludes from both sum and count); it does NOT treat NaN as 0.
(j)	T	<code>iloc</code> is position-based; <code>loc</code> is label-based. This is correct.

Common Pitfall / Exam Trap

Problem 1 tests 8 out of 10 statements as **False**. The exam heavily targets common misconceptions. When in doubt, the statement is likely false if it makes an absolute claim (e.g., “completely inaccessible,” “guaranteed to maintain order”).

Problem 2: Multiple Choice — Answers and Analysis

Q	Ans	Key Reasoning
(a)	C	<code>extend("34")</code> iterates the string: characters "3" and "4" are added separately.
(b)	B	Mutable default <code>y=[]</code> persists. Calls: <code>[1]</code> , <code>[1,2]</code> , <code>[3]</code> (new list passed), <code>[1,2,4]</code> (back to default).
(c)	B	Lambda captures <code>i</code> by reference. At call time, <code>i=2</code> for all lambdas, so all return <code>2*2=4</code> .
(d)	D	<code>&</code> is intersection; <code> </code> is union; <code>^</code> is symmetric difference; <code>-</code> is difference.
(e)	D	Assigning <code>my_dog.legs = 3</code> creates a new instance attribute that shadows the class attribute.
(f)	B	<code>a</code> has shape <code>(3,)</code> ; <code>b[:, None]</code> has shape <code>(3,1)</code> ; result broadcasts to <code>(3,3)</code> .
(g)	C	<code>df['Column_A']</code> returns a <code>pandas.Series</code> .

Problem 3: Debugging — Error Analysis

The debugging problem tests the ability to identify runtime and logical errors in a complete program.

Error	Line	Explanation
1	4	<code>len(total)</code> should be <code>len(grades)</code> . <code>total</code> is an integer (result of <code>sum()</code>), which has no <code>len()</code> , causing <code>TypeError</code> .
2	31	Charlie's grades are a string "90, 85, 92" instead of a list. <code>sum()</code> on a string causes <code>TypeError</code> .
3	32	David's grades list is empty <code>[]</code> , causing <code>ZeroDivisionError</code> when dividing by <code>len(grades) = 0</code> .

Exam Tip

Debugging questions require tracing execution order. The program processes students in dictionary iteration order. Alice and Bob would succeed (if line 4 were fixed), but Charlie and David trigger different errors. Focus on: (1) type mismatches, (2) wrong variable names, (3) edge cases like empty inputs.

Problem 4: Recursion — Permutation Pattern

The exam tests the **swap-based backtracking** approach for generating all permutations.

```

1 def recursive_permutations(chars, index):
2     # Base case: all positions fixed
3     if index == len(chars):
4         all_results.append("".join(chars))
5         return
6
7     # Recursive case: try each remaining character at position index
8     for i in range(index, len(chars)):
9         chars[index], chars[i] = chars[i], chars[index]    # choose
10        recursive_permutations(chars, index + 1)            # explore
11        chars[index], chars[i] = chars[i], chars[index]    # undo (
                        backtrack)

```

Key Point

Key elements to memorize:

1. Base case: `index == len(chars)`.
2. Loop range: `range(index, len(chars))`, not `range(len(chars))`.
3. Swap before recursion, swap back after (backtracking).
4. Use `"".join(chars)` to convert list back to string.
5. The inner function accesses `all_results` from the enclosing scope (closure).

Problem 5: OOP Fill-in-the-Blank — Pattern

Expected output:

```
Error: System not initialized.
Loading GPT-4...
[GPT-4 (1T parameters)] Generating: Write a poem about Python with a
window of 1000
```

Part	Answer	Concept Tested
(i)	<code>LanguageModel</code>	Inheritance: subclass declaration syntax.
(ii)	<code>super().__init__(name, params)</code>	Calling parent constructor via <code>super()</code> .
(iii)	<code>if not self._is_ready:</code>	Checking a protected attribute (single underscore).
(iv)	<code>self.model_info</code>	Accessing a <code>@property</code> without parentheses.
(v)	<code>gpt.load()</code>	Calling a method that sets <code>_is_ready = True</code> .

Common Pitfall / Exam Trap

Note that `model_info` is a `@property`, so it is accessed as `self.model_info` (no parentheses), not `self.model_info()`. Also, `_is_ready` uses a single underscore (protected convention), so it is accessible from the subclass.

Problem 6: NumPy Vectorization — Key Insights**Part (a): Correct Expression**

```
R = np.std(np.mean(np.where(X > c[np.newaxis, :],
                             X * np.sin(X),
                             X * np.cos(X)), axis=0))
```

Shape analysis:

1. `X`: shape (N, D).
2. `c`: shape (D,); after `c[np.newaxis, :]`: shape (1, D).
3. `X > c[np.newaxis, :]`: broadcasts to shape (N, D).
4. `np.where(...)`: shape (N, D).
5. `np.mean(..., axis=0)`: shape (D,) (collapse N samples).
6. `np.std(...)`: scalar (std over D features).

Part (b): Why the Naive Version Works Here**Common Pitfall / Exam Trap**

The sample solution argues that `X > c` can fail when `c` is a 2D column vector. However, in the problem setup, `c = np.random.randn(D)` has shape `(D,)`, which already broadcasts correctly against `X` of shape `(N, D)` without needing `np.newaxis`. The key point the exam tests is: always make broadcasting intent explicit using `np.newaxis` or `reshape` to avoid ambiguity when shapes could be misinterpreted.

Problem 7: Pandas Output Prediction**Part (a)**

```
df.loc[df['Points'] == df['Points'].max(), 'Player']
```

Step-by-step:

1. `df['Points'].max()` → 25.
2. `df['Points'] == 25` → boolean mask `[False, False, False, True]`.
3. `df.loc[mask, 'Player']` → Series with index 3, value 'David'.

Output:

```
3      David
Name: Player, dtype: object
```

Part (b)

```
result = df.groupby(by='Class', as_index=False).sum()
print(result.iloc[1, 0])
```

Step-by-step:

1. `groupby('Class', as_index=False).sum()` produces a DataFrame with `Class` as a regular column.
2. Groups: A (Score1: 30, Score2: 10), B (Score1: 70, Score2: 10).
3. `result.iloc[1, 0]` → row 1, column 0 → 'B'.

Output: B

Exam Tip

For Pandas output prediction:

1. Write out the intermediate DataFrame after each operation.
2. Pay attention to whether `as_index=True` (default) or `False`.
3. Remember that `loc` returns a Series (with original index) when selecting a single column.
4. Check the dtype line in Series output.

Topic Frequency and Priority

Topic	Questions	Study Priority
Mutable default arguments	Q1d, Q2b	High — appears in both T/F and MC
Set/Dict ordering and hashability	Q1b, Q1c, Q2d	High — fundamental misconceptions
NumPy broadcasting and shapes	Q1f, Q2f, Q6	Very High — large point value
NumPy views vs copies	Q1g, Q6	High
Lambda closure trap	Q2c	Medium — classic exam question
OOP: class vs instance attributes	Q2e, Q5	High
OOP: inheritance and <code>super()</code>	Q5	High
@property access syntax	Q5	Medium
Pandas <code>loc/iloc</code>	Q1j, Q7	High
Pandas <code>groupby</code> and <code>as_index</code>	Q7b	High
Pandas <code>mean()</code> skips NaN	Q1i	Medium
Recursion (backtracking)	Q4	Very High — 10 points for one function
Debugging (type errors, edge cases)	Q3	High — requires careful tracing

Exam Strategy Recommendations

Key Point

Time management for a 2-hour exam with 7 problems:

1. Problems 1–2 (T/F + MC): Target 15 minutes. These test breadth of knowledge.
2. Problem 3 (Debugging): Target 10 minutes. Read code line-by-line; check types.
3. Problem 4 (Recursion): Target 20 minutes. Memorize the swap-backtrack pattern.
4. Problem 5 (OOP): Target 15 minutes. Trace the expected output to determine each blank.
5. Problem 6 (NumPy): Target 25 minutes. Write shape annotations at each step.
6. Problem 7 (Pandas): Target 15 minutes. Build intermediate DataFrames on scratch paper.
7. Reserve 20 minutes for review and checking.

Exam Tip

On the cheat sheet, prioritize:

- The permutation backtracking template (Problem 4 pattern).
- NumPy broadcasting rules and shape arithmetic.
- `np.where` syntax and axis conventions.
- Pandas `groupby` with `as_index`, `agg`, `transform`.
- OOP: `super().__init__()`, `@property`, name mangling.
- The mutable default argument trap with `fix` pattern.
- Lambda closure trap with `fix (i=i)`.